

Dạng Tấn Quý

Mục lục

Đặng Tấn Quý

Lời nói đầu

Tài liệu này tổng hợp và viết lại nội dung **Chương 3.1 — Linear Regression** trong sách *Dive into Deep Learning* (Aston Zhang, Zachary C. Lipton, Mu Li, Alexander J. Smola), dựa trên:

- Bản tiếng Anh: https://d2l.ai/chapter_linear-regression/linear-regression.html
- Bản tiếng Việt: https://vi.d2l.ai/chapter_linear-networks/linear-regression.html

Mục tiêu: trình bày đầy đủ lý thuyết hồi quy tuyến tính — từ mô hình, hàm mất mát, nghiệm phân tích, gradient descent, vector hóa, liên hệ phân phối chuẩn, đến cách nhìn mạng nơ-ron một lớp — theo cùng phong cách giáo án (động cơ → định nghĩa → công thức → ví dụ → tổng kết).

Đặng Tấn Quý

1 Hồi quy và bài toán dự đoán số

Động cơ — Khi nào cần hồi quy?

Hồi quy (regression) là tập các phương pháp mô hình hóa mối quan hệ giữa một hoặc nhiều biến độc lập và một biến phụ thuộc. Trong khoa học tự nhiên và xã hội, mục đích thường là *mô tả* (characterize) mối liên hệ; trong **học máy**, mục đích thường là *dự đoán*. Bài toán hồi quy xuất hiện khi ta muốn dự đoán một **giá trị số**: giá nhà, giá cổ phiếu, thời gian nằm viện, nhu cầu bán lẻ, ... Không phải mọi bài toán dự đoán đều là hồi quy cổ điển — sau này ta sẽ gặp **phân loại** (classification), nơi mục tiêu là dự đoán nhãn thuộc một tập hạng mục.

1.1 Ví dụ chạy: dự đoán giá nhà

Giả sử ta muốn ước lượng **giá nhà** (USD) từ **diện tích** (ft²) và **tuổi nhà** (năm). Để xây mô hình, cần tập dữ liệu gồm giá bán, diện tích, tuổi của từng căn đã bán.

Thuật ngữ học máy trong hồi quy

- **Tập huấn luyện** (training set / training dataset): toàn bộ dữ liệu dùng để học mô hình.
- **Mẫu / ví dụ** (example, data point, instance, sample): một hàng dữ liệu (một lần bán nhà).
- **Nhãn / mục tiêu** (label, target): đại lượng cần dự đoán (giá nhà) — ký hiệu y .
- **Đặc trưng / biến giải thích** (feature, covariate): biến đầu vào (diện tích, tuổi) — vector $\mathbf{x}$.

Ký hiệu: n = số mẫu; mẫu thứ i : $\mathbf{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}, \dots]^\top$, nhãn $y^{(i)}$.

2 Các yếu tố cơ bản của hồi quy tuyến tính

Hồi quy tuyến tính là gì?

Hồi quy tuyến tính (linear regression) là công cụ **đơn giản nhất và phổ biến nhất** cho bài toán hồi quy, có từ đầu thế kỷ 19 (Gauss, 1809; Legendre, 1805). Hai giả định cốt lõi:

1. Mọi quan hệ giữa $\mathbf{x}$ và y **xấp xỉ tuyến tính**: kỳ vọng có điều kiện $E[Y | X = \mathbf{x}]$ biểu diễn được dưới dạng tổng có trọng số của các đặc trưng.
2. **Nhiều** quan sát được cư xử “đẹp” — thường giả định phân phối Gaussian.

2.1 Mô hình tuyến tính

Mô hình affine / tuyến tính

Giả định tuyến tính nghĩa là giá (mục tiêu) là tổng có trọng số của các đặc trưng:

$$\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b. \quad (1)$$

$w_{\text{area}}, w_{\text{age}}$ là **trọng số** (weights); b là **hệ số chặn** (bias, offset, intercept). Trọng số đo mức ảnh hưởng của từng đặc trưng; bias cho giá trị dự đoán khi mọi đặc trưng bằng 0 (dù thực tế không có nhà diện tích 0, bias vẫn cần để biểu diễn mọi hàm affine, không chỉ đường thẳng qua gốc).

Công thức (??) là **phép biến đổi affine**: tổng có trọng số tuyến tính cộng hằng số dịch chuyển.

Với d đặc trưng, dự đoán $\hat{y}$ (“mũ” = ước lượng):

$$\hat{y} = w_1 x_1 + \dots + w_d x_d + b, \quad (2)$$

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b, \quad \mathbf{w}, \mathbf{x} \in \mathbb{R}^d. \quad (3)$$

Ma trận thiết kế

Gom n mẫu vào **ma trận thiết kế** $\mathbf{X} \in \mathbb{R}^{n \times d}$: mỗi *hàng* là một mẫu, mỗi *cột* là một đặc trưng. Vector nhãn $\mathbf{y} \in \mathbb{R}^n$, vector dự đoán $\hat{\mathbf{y}} \in \mathbb{R}^n$:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b \quad (4)$$

(broadcasting khi cộng b vào từng hàng). Mục tiêu: chọn $\mathbf{w}, b$ sao cho trên dữ liệu mới cùng phân phối với $\mathbf{X}$, dự đoán nhãn có sai số kỳ vọng nhỏ nhất.

Nhiều đo lường

Dù quan hệ thật là tuyến tính, hiếm khi có $y^{(i)} = \mathbf{w}^\top \mathbf{x}^{(i)} + b$ chính xác với mọi i — sai số đo, nhiễu môi trường. Ta mô hình hóa:

$$y^{(i)} = \mathbf{w}^\top \mathbf{x}^{(i)} + b + \epsilon^{(i)}.$$

Trước khi tối ưu cần: (i) đo chất lượng mô hình; (ii) quy trình cập nhật tham số.

2.2 Hàm mất mát (loss function)

Sai số bình phương

Hàm mất đo khoảng cách giữa dự đoán và nhãn thật; giá trị không âm, dự đoán hoàn hảo $\Rightarrow \text{loss} = 0$. Với hồi quy, phổ biến nhất là **sai số bình phương** (squared error). Mẫu i :

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{2} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)})^2. \quad (5)$$

Hệ số $\frac{1}{2}$ tiện khi lấy đạo hàm (triệt tiêu hệ số 2). Trên toàn tập n mẫu:

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n l^{(i)}(\mathbf{w}, b) = \frac{1}{2n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)})^2. \quad (6)$$

Tính bậc hai của loss

Sai lệch lớn bị **phạt nặng hơn** (bình phương) — khuyến khích tránh lỗi lớn, nhưng cũng nhạy với điểm ngoại lai (outlier). Hình minh họa trên dữ liệu một chiều: đường thẳng “cắt giữa” các điểm, các đoạn thẳng đứng thể hiện sai số từng điểm.

Bài toán tối ưu

Tìm tham số tối ưu:

$$\mathbf{w}^*, b^* = \underset{\mathbf{w}, b}{\operatorname{argmin}} L(\mathbf{w}, b). \quad (7)$$

2.3 Nghiệm phân tích (analytic solution)

Công thức bình phương tối thiểu (normal equation)

Gộp bias vào $\mathbf{w}$ bằng cách thêm cột toàn 1 vào $\mathbf{X}$. Bài toán: $\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$. Nếu $\mathbf{X}$ full rank (các cột độc lập tuyến tính), đạo hàm bằng 0 cho:

$$\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y} \quad \Rightarrow \quad \mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (8)$$

Nghiệm duy nhất khi $\mathbf{X}^\top \mathbf{X}$ khả nghịch.

Hạn chế nghiệm phân tích

Nghiệm phân tích hiếm gặp trong học sâu — điều kiện quá hạn chế. Tuy nhiên với hồi quy tuyến tính, công thức (8) là nền tảng lý thuyết quan trọng.

2.4 Minibatch stochastic gradient descent

Gradient descent và biến thể

Khi không có nghiệm đóng, ta **lặp** cập nhật tham số theo hướng **giảm** loss — thuật toán **gradient descent** (GD), nền tảng tối ưu hóa học sâu.

- **Full batch GD**: đạo hàm trên *toàn bộ* tập — chậm, một epoch mới cập nhật một lần.
- **SGD**: mỗi bước chỉ một mẫu — nhanh về mặt lý thuyết thống kê nhưng kém hiệu quả tính toán (không tận dụng song song ma trận-vector).
- **Minibatch SGD**: mỗi bước lấy ngẫu nhiên tập con $\mathcal{B}_t$, $|\mathcal{B}|$ thường 32–256 (bội số lũy thừa 2). Cân bằng tốc độ phần cứng và ổn định thống kê; một số lớp (batch normalization) cần $|\mathcal{B}| > 1$.

Mỗi vòng lặp t , lấy minibatch $\mathcal{B}_t$, cập nhật:

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b), \quad (9)$$

η = **learning rate** (tốc độ học). Với loss bậc hai và mô hình affine, gradient có dạng đóng:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}), \quad (10)$$

$$b \leftarrow b - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}). \quad (11)$$

Hyperparameter

$|\mathcal{B}|$ và η do người dùng chọn, **không** học từ vòng lặp training — gọi là **siêu tham số** (hyperparameter). Có thể tinh chỉnh tự động (Bayesian optimization) hoặc thử công trên **tập xác thực** (validation set).

Sau T vòng (hoặc đến khi dừng), ghi nhận $\hat{\mathbf{w}}, \hat{b}$. Ngay cả khi dữ liệu tuyến tính không nhiều, tham số thu được thường **không** đúng bằng nghiệm tối ưu lý thuyết (hội tụ hữu hạn bước, minibatch ngẫu nhiên).

Tối thiểu toàn cục và khái quát hóa

Hồi quy tuyến tính (khi $\mathbf{X}^\top \mathbf{X}$ khả nghịch) có **một cực tiểu toàn cục**. Mạng sâu có nhiều điểm yên và cực tiểu cục bộ. Thực tế, ta thường chỉ cần tham số cho dự đoán **chính xác trên dữ liệu mới** — thách thức **khái quát hóa** (generalization), không phải fit hoàn hảo tập huấn luyện.

2.5 Dự đoán (prediction)

Với mô hình đã học $\hat{\mathbf{w}}^\top \mathbf{x} + \hat{b}$, ta dự đoán giá nhà mới từ diện tích x_1 , tuổi x_2 . Trong học sâu, giai đoạn này đôi khi gọi **inference**; trong thống kê, “inference” thường chỉ **ước lượng tham số**. Tài liệu dùng thuật ngữ **prediction** để tránh nhầm lẫn.

3 Vector hóa cho tốc độ

Tại sao vector hóa?

Khi huấn luyện, ta xử lý **cả minibatch** cùng lúc. Viết vòng `for` Python trên từng phần tử chậm hơn **rất nhiều** so với phép cộng vector hóa (+) gọi thư viện đại số tuyến tính (NumPy, PyTorch, ...).

Ví dụ: hai vector 10 000 chiều toàn 1.

- Cộng từng phần tử bằng `for`: thứ tự giây (tùy framework).
- `d = a + b`: thứ tự **milli giây** — tăng tốc hàng trăm đến hàng nghìn lần.

Vector hóa đẩy phép toán xuống thư viện tối ưu (BLAS), giảm lỗi lập trình và tăng khả năng tái sử dụng mã.

Lớp Timer (D2L)

Sách D2L dùng lớp `Timer` để đo nhiều lần chạy: `start()`, `stop()`, `avg()`, `sum()`, `cumsum()` — tiện so sánh `for-loop` và phép toán vector.

4 Phân phối chuẩn và loss bình phương

Động lực xác suất cho squared loss

Ngoài lý do thực tiễn (ước lượng $E[Y | X]$ khi quan hệ tuyến tính, phạt outlier), ta có thể **giả định xác suất** cho nhiều. Gauss (và Legendre) đặt nền hồi quy tuyến tính; phân phối chuẩn (Gaussian) gắn chặt với squared loss.

4.1 Phân phối chuẩn

Mật độ Gaussian

Phân phối chuẩn với trung bình μ , phương sai σ^2 (độ lệch chuẩn σ):

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x - \mu)^2\right). \quad (12)$$

Đổi $\mu \Rightarrow$ dịch theo trục x ; tăng $\sigma \Rightarrow$ phân phối dẹt, đỉnh thấp hơn.

4.2 Mô hình nhiễu Gaussian và MLE

Giả sử quan sát có nhiễu cộng Gaussian:

$$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2). \quad (13)$$

Likelihood của y cho $\mathbf{x}$ cố định:

$$P(y | \mathbf{x}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2} (y - \mathbf{w}^\top \mathbf{x} - b)^2\right). \quad (14)$$

Trên toàn dữ liệu (các cặp độc lập):

$$P(\mathbf{y} | \mathbf{X}) = \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)}). \quad (15)$$

Theo **nguyên lý hợp lý cực đại** (maximum likelihood), chọn $\mathbf{w}, b$ tối đa hóa $P(\mathbf{y} | \mathbf{X})$. Tương đương **tối thiểu hóa negative log-likelihood**:

$$-\log P(\mathbf{y} | \mathbf{X}) = \sum_{i=1}^n \frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} (y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b)^2. \quad (16)$$

Nếu σ cố định, số hạng đầu không phụ thuộc $\mathbf{w}, b$; số hạng thứ hai trùng (nhân hằng số) với **MSE** trong (??). Vậy:

MSE $\Leftrightarrow$ MLE dưới nhiễu Gaussian

Tối thiểu hóa **trung bình sai số bình phương** tương đương ước lượng hợp lý cực đại mô hình tuyến tính với giả thiết nhiễu cộng Gaussian. Nghiệm không phụ thuộc σ .

5 Hồi quy tuyến tính như mạng nơ-ron

Mô hình tuyến tính trong ngôn ngữ mạng nơ-ron

Mạng nơ-ron đủ biểu đạt để **bào hàm** (subsume) mô hình tuyến tính: mỗi đặc trưng là một nơ-ron đầu vào, nối thẳng tới đầu ra. Sơ đồ nhấn mạnh **topology** (ai nối ai), không vẽ giá trị cụ thể của trọng số.

Mạng một lớp kết nối đầy đủ

Đầu vào $x_1, \dots, x_d \Rightarrow$ số chiều đặc trưng d . Đầu ra o_1 (một giá trị số) $\Rightarrow$ một nơ-ron đầu ra. Giá trị đầu vào **cho trước**; chỉ có một nơ-ron **tính toán**. Thường **không đếm lớp đầu vào** khi nói số lớp $\Rightarrow$ hồi quy tuyến tính là mạng **một lớp** (single-layer), **kết nối đầy đủ** (fully connected / dense) từ đặc trưng tới đầu ra.

5.1 Sinh học: nơ-ron thật và McCulloch–Pitts

Liên hệ sinh học (không phải động lực chính ngày nay)

Hồi quy tuyến tính (1795) trước khoa học thần kinh tính toán, nhưng McCulloch và Pitts (1940s) mô hình hóa nơ-ron nhân tạo tương tự nơ-ron sinh học:

- **Dendrite**: nhận tín hiệu x_i từ nơ-ron khác hoặc cảm biến.
- **Synaptic weights** w_i : trọng số hóa ảnh hưởng ($x_i w_i$).
- **Nucleus**: tổng hợp $y = \sum_i x_i w_i + b$.
- **Axon**: truyền y (có thể qua hàm phi tuyến $\sigma(y)$) tới cơ bắp hoặc nơ-ron khác.

Ý tưởng: nhiều đơn vị kết hợp đúng cách có thể tạo hành vi phức tạp hơn một nơ-ron. Tuy nhiên, học sâu hiện đại lấy cảm hứng mạnh từ toán, thống kê, khoa học máy tính — Russell & Norvig (2016): máy bay lấy cảm hứng từ chim nhưng điều học không còn là động lực chính của hàng không.

6 Tổng kết

Chương 3.1 — Những điểm cần nhớ

1. **Hồi quy tuyến tính**: chọn $\mathbf{w}, b$ để tối thiểu squared loss trên tập huấn luyện; mô hình $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$.
2. **Thành phần học máy**: dữ liệu, mô hình, hàm mất mát, thuật toán tối ưu (thường minibatch SGD).
3. **Nghiệm phân tích**: $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ khi khả nghịch.
4. **Vector hóa**: nhanh hơn và ít lỗi hơn vòng lặp Python.
5. **MSE** $\Leftrightarrow$ MLE với nhiễu Gaussian cộng.
6. **Mạng nơ-ron**: hồi quy tuyến tính = một lớp kết nối đầy đủ; là bước đệm tới mạng sâu (parametric form, loss khả vi, SGD, đánh giá trên dữ liệu mới).

7 Bài tập (theo D2L 3.1.6)

- Cho dữ liệu $x_1, \dots, x_n \in \mathbb{R}$. Tìm b tối thiểu hóa $\sum_i (x_i - b)^2$.
 - Tìm nghiệm phân tích của b .
 - Liên hệ bài toán này với phân phối chuẩn.
 - Nếu đổi loss thành $\sum_i |x_i - b|$ (L1), có nghiệm đóng không?
- Chứng minh các hàm affine $\mathbf{x}^\top \mathbf{w} + b$ tương đương hàm tuyến tính trên vector mở rộng $(\mathbf{x}, 1)$.
- Muốn mô hình hóa hàm bậc hai

$$f(\mathbf{x}) = b + \sum_i w_i x_i + \sum_{j \leq i} w_{ij} x_i x_j.$$

Biểu diễn trong mạng sâu / mở rộng đặc trưng như thế nào?

- Điều kiện khả nghịch $\mathbf{X}^\top \mathbf{X}$ (full rank):
 - Nếu không full rank thì sao?
 - Cách khắc phục? Thêm nhiều Gaussian nhỏ độc lập vào mọi phần tử $\mathbf{X}$ thì kỳ vọng của $\mathbf{X}^\top \mathbf{X}$ thay đổi thế nào?
 - SGD khi $\mathbf{X}^\top \mathbf{X}$ không full rank?
- Giả sử nhiễu ϵ có phân phối Laplace (exponential bậc hai): $p(\epsilon) = \frac{1}{2} \exp(-|\epsilon|)$.
 - Viết $-\log P(\mathbf{y} | \mathbf{X})$.
 - Có nghiệm đóng không?
 - Đề xuất minibatch SGD; điều gì có thể sai gần điểm dừng? Cách sửa?
- Ghép hai lớp tuyến tính liên tiếp (đầu ra lớp 1 là đầu vào lớp 2). Tại sao tổ hợp “ngây thơ” này không tăng biểu đạt lực?
- Hồi quy giá nhà / cổ phiếu thực tế:
 - Giả thiết nhiễu Gaussian cộng có phù hợp không? (giá âm? biến động?)
 - Vì sao hồi quy $\log(\text{price})$ thường tốt hơn?
 - Cổ phiếu giá rất thấp (penny stock): vấn đề gì? (Black–Scholes, 1973)
- Dự đoán số táo bán trong siêu thị:
 - Vấn đề của mô hình Gaussian cộng (bán táo, không phải dầu).
 - Phân phối Poisson: $p(k | \lambda) = \lambda^k e^{-\lambda} / k!$. Chứng minh $E[k] = \lambda$.
 - Thiết kế loss gắn với Poisson.
 - Thiết kế loss cho ước lượng $\log \lambda$.
- (Bổ sung bản Vi)** Viết bài toán tối ưu hồi quy tuyến tính dạng ma trận–vector; tính gradient theo $\mathbf{w}$; giải $\nabla_{\mathbf{w}} L = 0$. Khi nào tốt hơn SGD? Khi nào phương pháp ma trận “vỡ”?

Nguồn: Aston Zhang et al., Dive into Deep Learning, §3.1. Bản LaTeX biên soạn theo template Nhóm 5 — Xác suất và Thống kê.